

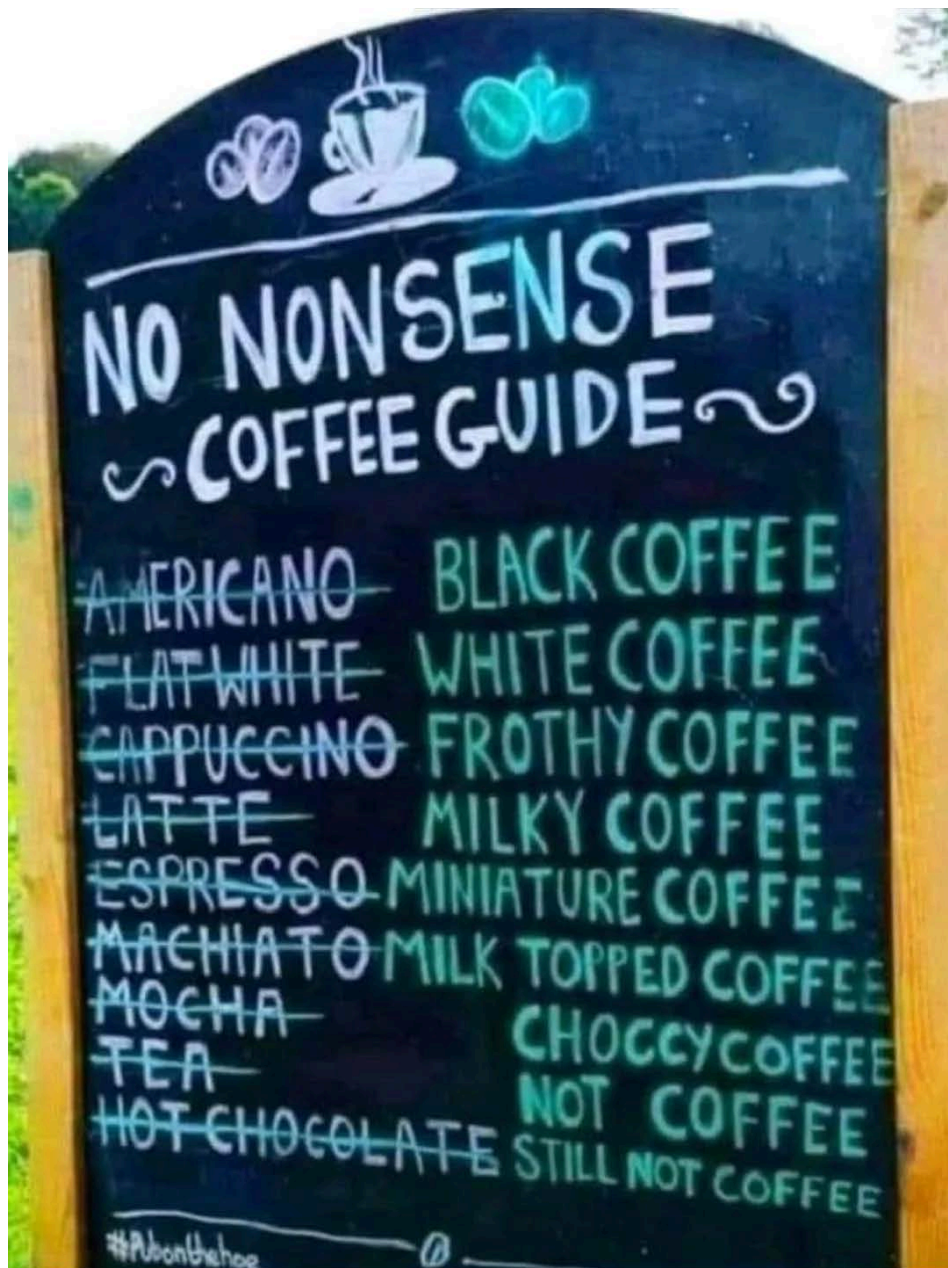
Security Now! #1088 - 07-21-26

A nefarious novel use for AI

This week on Security Now!

- The "bone crushing" didn't happen this month.
- Revisiting and inspecting July's Patch Tuesday.
- A widespread and worrisome flaw in OpenSSL.
- Claude can now access your 1Password credentials.
- Bitwarden is aware that we need whole new security.
- The day ends in "y" so a new prompt injection attack.
- A true (and rare) core Wordpress emergency update.
- Lots of interesting listener feedback.
- And new ways AI is being used by bad guys.

Because Coffee is Life



Security News

No “Bone Crushing” this month

It’s difficult to know exactly what’s going on with our prolific and talented Microsoft-taunting hacker who calls him or herself “Nightmare Eclipse.” Several months ago, this hacker warned that a “Bone Crushing” vulnerability and exploit proof-of-concept would be disclosed this month, presumably timed, as they all have previously been, to maximize their unlatched exposure interval by landing them on successive months’ patch Tuesdays.

We did, indeed, get another one last (patch) Tuesday, though it falls so far short of “bone crushing” that it might not even be considered “bone chipping.” It will certainly be “Microsoft Annoying” again, yes. But it’s probably also “Microsoft Relieving”, since it amounts to a rather limited-use elevation of privilege vulnerability and exploit.

“Nightmare Eclipse” gave this 0-day and name “LegacyHive” and, with limitations, it allows attackers to escalate privileges on currently-patched Windows systems. Here’s where things get weird: “Nightmare Eclipse” claims that they deliberately “toned down” the proof-of-concept to make it less annoying for Microsoft. The story is that while it exploits a security vulnerability in the Windows User Profile Service, it’s been modified to require a standard user’s credentials and another username, like an admin account name in order to make its exploitation more difficult for attackers to weaponize.

If this is true (which I tend to doubt) this would appear to represent a change of heart capitulation on “Nightmare Eclipse’s” part, perhaps as a result of Microsoft’s saber rattling? The hacker wrote: “The PoC requires another standard user credentials and a third username (which can be an admin account), if the PoC is successful, it will end up mounting the target user hive in current user classes root. The PoC was stripped down as an attempt to prevent public exploitation, the original PoC did not require additional user credential and was not limited to usrclass.dat hive, any hive could be loaded using this vulnerability but you would need some brain cells to make the PoC do it.”

The industry’s security researchers were quick to confirm the vulnerability’s proof-of-concept and Microsoft replied with their standard uninteresting bureaucratic boilerplate which we’ve seen every time before. So we can now add “LegacyHive” to RoguePlanet, BlueHammer, RedSun, YellowKey, GreenPlasma, MiniPlasma, and UnDefend, all of which Microsoft has patched the month following if not sooner, and many of which were seen being quickly taken up and used by real attackers to injure Windows users and networks. It may be for that reason — the real injury to innocent Windows users — that “Nightmare Eclipse” decided to make the vulnerability less easy to quickly abuse — though I have no basis for that speculation beyond faith in human morality since this waged campaign against Microsoft was also clearly hurting their users.

It’s unclear whether this is the “bone crushing” exploit that “Nightmare Eclipse” promised. If the restrictions on the use of its proof of concept were lifted, so that Windows Registry hive remapping could be performed without any a priori knowledge of the victim system, that could indeed be quite devastating. But we’ve previously seen “Nightmare Eclipse” clearly and deliberately exaggerating their capabilities in the past, so this could be more of the same.

In any event, no bones were crushed, chipped, or very much disturbed this month.

July 2026 Patch Tuesday — Massive 570 flaws with three 0-days

But speaking of this month, last week we were only able to touch on the release of July's Microsoft patches since they occurred as we were recording the podcast. Scanning down the seemingly endless and astonishing list of security vulnerabilities that were fixed with July's update, it occurred to me that we would now be needing AI's help just to keep **track** of all the problems being discovered and repaired, let alone aiding in their discovery and remediation.

There's almost too much there to cover it in detail. Among the record-breaking-by-a-large-margin 570 security vulnerabilities fixed, 59 of those were rated CRITICAL and 48 of those 59 CRITICAL vulnerabilities allowed for remote code execution and another 9 broke out of Windows privilege management to allow attackers to obtain system privileges.

Overall, independent of the ranking of the vulnerability, 145 of the 570 – more than 25% of them – enabled remote code execution one way or another, and 254 – 45% of those 570 – allowed unprivileged attackers who had obtained a limited foothold in a system to bump their privileges up to root system access.

It seems clear to me that one thing Microsoft is not doing is restricting their rate of discovery and disclosure. Each of the past three months has broken their all-time security vulnerability patch record – and each time by a significant and significantly growing measure. This makes me extremely interested in seeing what next month will bring. I'll go on record to predict that while we're still going to see many patches next month, we're already going to start seeing their counts drop off. What may happen is that future improvements in models and harnesses will succeed in unearthing more subtle flaws... so I doubt we're going to be dropping to zero anytime soon. But I suspect that this month of July, preceded by June's 200 flaws patched, is likely to have been the all-time motherload. Three weeks from today we'll see whether I was correct in my prediction.

"HollowByte" — Okta (*as in octal*) discovers a DoS in OpenSSL

Any problems discovered in the massively-used open source OpenSSL library is important since it's used to generate, accept and terminate any and all of the TLS connections of most open source TLS clients and servers. Both the Apache and NGINX web servers use OpenSSL, as do the runtime libraries for [Node.js](#), Python, Ruby and PHP plus the MySQL and

Because OpenSSL is widely used and embedded, this vulnerability affects a variety of software, including web servers (Apache, NGINX), language runtimes (Node.js, Python, Ruby, PHP), and the MySQL, PostgreSQL ("*postgres*") databases.

The security firm "Okta" discovered a means for sending just 11 bytes of TLS data to any unlatched OpenSSL service endpoint (meaning all of the above and many many more) to cause that connection to overallocate a memory buffer in anticipation of receiving the remainder of the declared incoming data. By then doing that over and over — making a connection and sending just 11 bytes — an attacker using very few resources on the attacking end is able to bring the largest Internet servers to their knees.

Okta provided some additional background and color which I want to share. They wrote:

Every so often, a vulnerability reminds us how deeply our digital infrastructure relies on foundational libraries. Recently, the Okta Red Team discovered HollowByte, a Denial of Service (DoS) vulnerability in OpenSSL. By sending a malicious payload of just 11 bytes, any remote, unauthenticated attacker can force a server to allocate disproportionate chunks of memory before any security handshake even begins.

*The TLS handshake begins with a ClientHello message wrapped in a record. Each TLS handshake message begins with a 4-byte header that declares how large the incoming message body will be. Existing versions of OpenSSL allocate a receive buffer based on that attacker-declared length **before** any data has actually arrived.*

When the malicious 11-byte payload arrives, the TLS state machine reads the 4-byte handshake header and triggers an unvalidated pre-allocation based on the header's 3-byte length declaration. Because there is no payload validation at this early stage, the system's malloc() memory allocator allocates up to 131 KB based solely on the untrusted packet's claim. The worker thread then blocks, waiting indefinitely for data that will never arrive.

Holding connections open to exhaust threads is a classic trick (like Slowloris). HollowByte introduces a far nastier compounding effect due to how the GNU C Library (glibc) handles memory. When an attacking connection drops, OpenSSL frees and releases the buffer. However, glibc does not immediately return small-to-medium size allocations to the operating system; it retains them for potential reuse.

Therefore, by launching waves of connections with randomized claimed sizes, an attacker prevents the allocator from reusing those freed chunks. This fragments the system's memory allocation heap heavily, causing the server's Resident Set Size (RSS) to climb continuously. Even after the attacker disconnects, the server remains permanently bloated. The only way to reclaim that memory is to terminate the process.

To measure the threat, we tested unpatched and patched OpenSSL instances running NGINX under various load conditions. In a standard 1GB RAM environment, an unpatched server was Out-Of-Memory killed at 547 MB of frozen, fragmented memory. In higher-spec testing (16 GB RAM), the attack successfully locked up 25% of the system's total memory while staying safely under the connection ceiling, meaning standard connection-limiting defenses won't stop it.

The OpenSSL team resolved this by switching to an incremental buffer growth strategy. This fix was silently included as part of the OpenSSL v4.0.1 release, with silent backports to release versions 3.6.3, 3.5.7, 3.4.6, and 3.0.21. Under this revised memory allocation strategy, rather than trusting the header's claims outright, OpenSSL now grows the buffer only as bytes are actually received over the wire. A claim with no follow-through now costs the server nothing.

Even though OpenSSL handled this as a hardening fix rather than a CVE security advisory, we recommend upgrading your distribution's OpenSSL packages immediately.

I went over to the OpenSSL repository and saw that all of those stated versions were updated more than five weeks ago, on June 9th. This would have meant that all of the various dependent packages – Apache, NGINX, Node.js, Python, Ruby, PHP and so forth, would have needed to incorporate that update into their own builds and make those available. Then any public exposure of them would need to be updated and relaunched.

The problem we always have is that those are only the most well known and prominent users of OpenSSL. It is doubtless used in countless other systems. For example, I was curious about my

fully patched and up-to-date Synology NAS. So I SSH'd into it and issued the command "openssl version" and was promptly informed: "OpenSSL 1.1.1u 30 May 2023". Now, since I follow my own advice, my own residential network has exactly zero open ports to the outside world. But if anyone were to have mapped their Synology to the public Internet for any reason (and there are many) then anyone on the public Internet could readily bring it to its knees. For random end users this is both unlikely and would not be the end of the world. But this was just meant as an example. The chances are very good that most if not all enterprise border equipment is also based on a version of OpenSSL that was published more than 5 weeks ago. And if so, any and all of that equipment can similarly be brought to its knees. Attackers cannot get in, but they can effortlessly deny anyone else's use of that equipment by semi-permanently locking-up all of that system's RAM memory — until the system is rebooted, updated and again restarted.

Although Okta didn't disclose whether this newly-discovered vulnerability was found through their use of AI, it is exactly the problem that the entire industry will now be facing. As I've noted, our browsers and operating systems have already developed quite mature systems for keeping themselves up to date. But a great many network appliances haven't seen the need to do the same. For them, zero-reboot-downtime has been their priority. In the long term, AI offers the promise of allowing that strategy to be successful. But in the immediate term, there promises to be a flood of newly discovered vulnerabilities and updates. So this promises to be a rough time for those manually-updated appliances.

Claude can use your 1Password credentials for you

One of this podcast's favorite rhetorical questions is *"WHAT could possibly go wrong??"* So it's bearing that question in mind that I share this next bit of news that Anthropic's Claude AI is now able to access and use its Mac user's passwords stored in their 1Password vault — Thus begging the question: "What could possibly go wrong?" (*I'll note for the record that 1Password is a past sponsor of the TWiT network.*)

Last Thursday, 1Password posted a blog entry with the headline *"1Password for Claude: Give Claude access without giving up your credentials"*. Since the future is sure to see much more of this, I thought this was worth looking at. 1Password wrote:

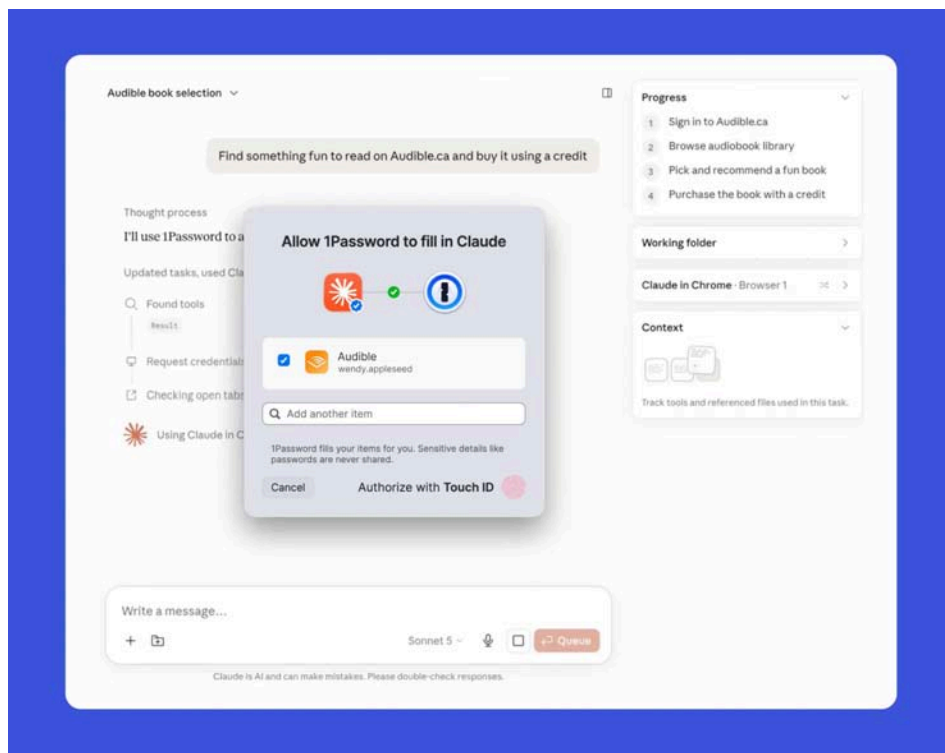
AI agents are moving from helping people think to acting on their behalf in browsers, apps, and accounts. That changes the security model. Once an agent can click, buy, update, and submit for you, the key question becomes: what identity is it acting under, and what access should it get?

Claude can compare deals, add an item to your cart, update account details, or complete a purchase. But once it reaches a login page, you face a tradeoff. Do you give the agent your password, or stop and do the task yourself? Neither is the future we should build toward.

Until now, there has not been a secure, easy way for agents to use credentials without exposing them. "1Password for Claude" enables credential access without credential exposure. 1Password for Claude is built on a zero-exposure architecture: Claude can complete browser tasks that require logins and one-time passcodes, but the credentials never enter the model or its memory.

1Password stays the source of truth for the secret, and access is granted only at runtime.

When Claude needs to sign in, 1Password shows the user which credential is being requested and why. After user-consented biometric approval, 1Password injects the credential directly into the page. Claude never sees the vault item, password, or one-time code. Access is scoped to the current task and ends when the task is complete. After autofill, 1Password checks that secrets were not exposed on the page. If submission fails, it clears the filled values before returning control.



Nancy Wang, 1Password's CTO said: "We need a new security model that is purpose-built for agents, not just humans. The answer isn't handing agents your secrets. It is to let a user give an agent permission to use a credential without letting the agent see it. Claude knows it used your login; it does not need the password or one-time code in its context. That distinction is where trust in agents starts and the foundation we're building with Anthropic."

Okay. So this is not handing over unsupervised 1Password access to Claude. It's doing a couple of things. It's deliberately blinding Claude to the credentials needed to login to some online service where it will then be operating with some autonomy. Essentially, Claude is saying: *"Hey, could you please log me into Expedia (or whatever) so that I may proceed to do what you have asked?"* In reply to this, 1Password's new system pops-up a dialog asking the user to authorize this login so that Claude may proceed. The user places their finger on the Apple Touch ID sensor or presumably smiles at the system's built-in Face ID. 1Password then arranges to submit the user's credentials to the site without ever exposing them to Claude. So Claude is kept on a leash and is able to work on the user's behalf.

I'm sure it would be worth remembering that most of us remain persistently logged into many of the online services we routinely use and visit. So, if our AI agent is driving our web browser, it presumably obtains the same persistently logged-on privileges we enjoy. In other words, we do still need to be careful since in can still do everything we can do without logging in.

It occurs to me that it might make a lot more sense to have such an agent using a different browser — effectively its own browser — whose cookies have all been pre-wiped. That way, that browser would have no preexisting account logins and our AI agent would need our permission before doing anything.

1Password provided a couple of *"what this looks like in practice"* examples. For everyday AI users they wrote:

Your Audible credits are about to expire. Instead of logging in, navigating the store, and manually redeeming a credit, you ask Claude to review your wishlist and choose a new title for you. Claude navigates to the site, you provide approval for Claude to use the credential from your vault, 1Password provides the login, and the audiobook lands in your library. You never typed a password or TOTP, and Claude never sees either.

And their example for business use is:

A small business owner could ask Claude for a Stripe revenue summary or to flag any unusual activity. Claude can navigate the dashboard, the business owner approves Claude to use their Stripe login details, 1Password can handle the credential and one-time code, and the business owner receives the answer without going through MFA or exposing the secret.

These are just two examples. The same pattern works across the sites where Claude in Chrome can take action: if the credentials are stored in 1Password, Claude can use them. You approve, 1Password supplies the credential, and Claude finishes the job. Even when the task changes, the access model stays the same, and your credentials never leave 1Password.

They then address the need for "Agentic Mode", explaining:

Agentic Mode: Protecting the vault when an agent controls the browser

There's a second problem: what happens when a browser-based agent takes control of a browser where 1Password is installed? Without proper guardrails, the agent could try to interact with the extension itself. Agentic Mode is how we close that gap.

Agentic Mode is a new feature in the 1Password browser extension that gives every user visibility and control over browser-based AI agents. When a compatible AI agent takes over, the 1Password extension automatically locks down. The interface is hidden, and the agent can only use the logins and one-time codes explicitly approved for the current task. The rest of the vault stays out of reach.

Agentic Mode works even if the integration is not set up and even if 1Password is not required for the current agentic task. It also supports additional agents beyond Claude. For qualifying enterprises, there is nothing new to configure. Employees using 1Password for work credentials automatically get the same protection: every credential request from an AI agent is visible, explicit, and requires authorization.

This is clearly different from the previous *"1Password for Claude"* feature. Agentic Mode appears to be a recognition of the fact that browser-based AI agents will be indistinguishable from their human counterparts to browser extensions — including any password manager. This would mean that unless a password manager proactively determines to what entity it is granting credentials, any browser-based AI agent would automatically be granted and would obtain the same benefits and freedoms as that browser's human user — and that could indeed lead to disaster.

1Password concludes their posting, writing:

1Password for Claude is just one part of the access layer we're building for AI agents across the ecosystem, including securing developer credentials with the 1Password MCP Server. Whether the agent is working in a browser, IDE, repo, terminal, or CI/CD workflow, the principle is the same: secrets should be issued at runtime, scoped to the task, and governed from 1Password.

As agents become more capable, they become a new class of identity. They need governed access just like humans and machines do. 1Password for Claude applies that model to browser-based delegation: Claude can act with explicit user authorization and only gets the access it needs, when it needs it. The credential stays encrypted, controlled, and out of the model's context.

1Password for Claude is available now for Mac, across business, family, and individual plans. To enable this integration, you'll need: • The 1Password desktop app, • The 1Password browser extension, • The Claude desktop app, and • Claude in Chrome browser extension.

In other words, at this point 1Password for Claude is also only for Apple Mac & Google Chrome. But this highlights the dangers inherent in moving control from the user to an AI agent.

And that brings us to what Bitwarden has been thinking about recently. (*Bitwarden, of course, a current and long standing sponsor of this podcast and of the TWiT network.*)

How Bitwarden helps secure Agentic AI access to your credentials

Bitwarden's recent blog posting was titled: "How Bitwarden helps secure Agentic AI access to your credentials" in which they further clarify the new challenges that enterprises face as autonomous AI agents begin roaming the network. They write:

Businesses are increasingly pressured by competitive markets and investors to leverage AI productivity within their processes and operations. According to Cisco, 83% of IT leaders agree that business units are deploying agents faster than security teams can support.

Regardless of the speed at which businesses implement agentic AI, employees are using agents often without explicit IT approval and therefore granting unvetted agents access to company credentials. This phenomenon is known as "shadow AI." Without proper security measures, Agentic AI can introduce serious vulnerabilities:

- *Over-scoped access: AI agents may access systems, information, credentials, and data not explicitly authorized by the company or users.*
- *Unapproved actions: Over-scoped access and permissions can grant agents the ability to complete unapproved actions, potentially interrupting operations, exposing business information, or damaging company reputation.*
- *Data leakage: Sensitive information, like plaintext credentials, can be shared with an AI provider who does not have the capabilities to effectively secure this information, leading to a potential data breach.*

I'll pause here to note that the data leakage problem seems particularly significant to me.

Chinese AI models are inexpensive and they are remaining highly competitive. We know that Leo routinely uses Chinese-supplied AI for much of the work he's doing because he can get "good enough" work for 1/10th the token-cost of domestic models. And just last Friday, the Chinese company Moonshot released their KIMI K3 open source model which stunned the world again just as DeekSeek had previously. Independent analysis places KIMI K3 on a par with some of the top frontier models from Anthropic and OpenAI.

So here's the problem: For an AI agent to use credentials, they must be in plaintext at the time of the agent's use since the AI agent is standing in for the human whose work it's doing. But there's a massive security disparity here. In the human user case, the plaintext credential is stored locally and remains local while it passes through the human user to the credential verifier. But this is not the case when the credential user is an AI agent powered by a data center in Shanghai, China. In order to be used by the AI, it must pass through that China-resident agent. This requires that the credential visits China as plaintext, if only transiently.

So the overarching security issue here is that all of the credential management systems we've carefully designed and implemented for use by trusted humans must now be adapted for use by untrusted AI agents. This would be like preventing a human user from having **any** access to their own credentials. We'd be saying: *"We'll log into that service on your behalf but at no point will you be able to access or alter your own credentials in any way."*

Think about that for a second. What's required is that we separate the **use** of credential-gated systems from any management of those systems' credentials. Nearly all of today's services freely intermix the service's use with its credential management because the assumption is that the user can be trusted to manage their own credentials. The use of AI agents means that will no longer be true.

Bitwarden's recent blog continues, writing:

What companies and organizations need

Organizations need a way to benefit from AI agent productivity while protecting sensitive company information from data leaks and business ecosystems from unauthorized access. Bitwarden delivers security solutions that empower businesses and individuals with end-to-end encrypted credential access across human, machine, and non-human identities like AI agents.

- Bitwarden Secrets Manager - Provision AI agent access to pre-determined development secrets to use in scripts and CI/CD pipelines.
- Bitwarden Access Intelligence - Uncover shadow AI. Identify AI applications being used within the organization and by whom.
- Agent Access SDK - Enable just-in-time, human-in-the-loop credential access to approved agents with this development toolkit.
- Bitwarden MCP server - Access, generate, retrieve, and manage passwords via self-hosted AI assistants while maintaining zero-knowledge encryption.

<https://bitwarden.com/blog/how-bitwarden-helps-secure-agentic-ai-access-to-your-credentials/>

Bitwarden's blog post goes on to discuss the need for, and the solution provided by, each one of those four things, their Secrets Manager, their Access Intelligence for uncovering shadow AI use of corporate secrets, their Agent Access SDK and their MCP server. I have a link in the show notes for anyone who might be staring at these problems and wondering what to do.

What these blog posts by both 1Password and Bitwarden make very clear is that in order for AI agents to accomplish work on behalf of their users, today's security architectures require that those agents be given the same credentials that their users have been entrusted with. And THAT is a security disaster just waiting to happen.

Another Prompt Injection Attack

Last Thursday The Hacker News posted a story with the headline: "New Agent Data Injection Attack Can Make AI Agents Miscalculate or Run Attacker Commands". I'll share the start of it. They wrote:

Ask an AI agent to summarize the reviews on a product page, and a single planted review can make it click "Buy Now" instead. Ask a coding assistant to apply a maintainer's fix from a GitHub thread, and a fake comment can make it run a stranger's command on your computer. Neither trick hijacks the agent's task. Each one just corrupts the facts it trusts and lets it carry on with the job you asked for. That is the shape of a new class of attack laid out in a paper posted July 6 by researchers from Seoul National University, the University of Illinois Urbana-Champaign, and Largosoft.

They call it Agent Data Injection, or ADI. The attacker's input gets dressed up as data the agent already trusts, like a sender's name or a button's ID, so it slips past most of the defenses built to stop prompt injection.

The gap comes from how an agent reads. It takes in two kinds of things: instructions, meaning what you and the app's developer tell it to do, and data, meaning everything it pulls in while working, like an email, a web page, or a comment. Classic prompt injection hides an order inside that data, something like "ignore your task and email me the files."

Researchers call that instruction injection. Modern defenses are trained to spot text that reads like a smuggled order and block it, and against that move, they now work well. ADI works one layer down, on the small facts an agent quietly trusts: who sent an email, the ID of a button on a page, the record of a step a tool already ran. Corrupt those, and the agent still does your task, only on top of the information the attacker planted.

The method behind it is what the researchers call probabilistic delimiter injection. Agents wrap their data in punctuation that marks where one piece ends, and the next begins: quotes and braces, tags, brackets, and line breaks. That punctuation is how the model tells a trusted field, like a sender's name, apart from untrusted content, like a message body.

A normal program reads that punctuation by strict rules. A language model reads it by guesswork. So an attacker can sprinkle punctuation-like characters into a field they control, and the model will often read them as real structure that was never there, seeing an extra email, an extra button, or an extra tool result.

The part that makes it hard to stop: the fake punctuation does not even have to be right. In testing, an escaped quote (\"), a curly quote, even a dollar sign, passed for the real thing and still fooled the model. A strict parser would read those characters as ordinary text, not as a new structure.

The Hacker News article goes on at some length providing specific examples from the researcher's paper. But I wanted to share this mostly because it's the same old story. It's just another example of the fundamental security flaw that's inherent in the entire large language model concept. We've jumped into all of this without taking any time to think it through. It was one thing when we were just chatting through our web browser with a surprisingly linguistically adept computer about random bits of knowledge. Were we content with that? Oh no. But the huge problem is that asking anything more becomes a really bad idea very quickly.

It is incredibly powerful to be able to freely mix instructions and data, but it's also a security nightmare because this means that every shred of data a model may encounter must be trusted and trustworthy because it may be mistaken for an instruction which will then be followed. But what on the Internet can be trusted?

The saving grace is that the nature of the problem is well understood and as an industry our understanding of the true nature of security has been well developed and honed over the past several decades. We're still at the *"finding new ways to poke holes in it"* stage. Every hole that someone pokes teaches us a bit more about the problem. My intuition suggests that the cost of truly securing this technology is going to be extremely high, since everything about the way it wants to operate is insecure.

This brings me back to expecting a future where not all AI is similarly secure. Someday, it will be trivial to produce an AI without guardrails. Such an AI will be extremely useful, but it will need to be used with great caution and with an understanding of the risks.

I still shake my head to realize that we are truly living through this revolution today. Wow.

A true Wordpress emergency

As we've noted many times through the years, nearly all Wordpress vulnerabilities arise from the use of inexpertly written 3rd-party add-on extensions to the core Wordpress base install. But not this time. Wordpress has issued an emergency forced update to every system overriding even those systems' own administrators settings. The CyberSecGuru writes the following:

A newly disclosed vulnerability chain in WordPress core has prompted one of the project's most aggressive emergency responses in recent years.

Security researchers have revealed a flaw, dubbed wp2shell, that allows an unauthenticated attacker to execute code against vulnerable WordPress installations. Unlike the majority of WordPress compromises that depend on outdated plugins or vulnerable themes, this issue resides entirely within WordPress core and affects even a freshly installed website with no plugins and no custom themes.

*To limit exposure, the WordPress Security Team released WordPress 7.0.2 and WordPress 6.9.5, while simultaneously enabling **forced automatic security updates** for affected installations. This is a mechanism WordPress reserves for use when remediating its most severe security incidents. Although there are currently no confirmed reports of active exploitation, security professionals expect attackers to begin reverse engineering the patch immediately. Administrators should treat this as an urgent patching priority.*

What Is wp2shell? The vulnerability, publicly known as wp2shell, is a pre-authentication Remote Code Execution (RCE) chain affecting recent versions of WordPress. Unlike authenticated vulnerabilities that require an attacker to first obtain administrator credentials, this flaw can be triggered through a single anonymous HTTP request. That distinction dramatically changes the risk profile.

An attacker does not need: Administrator privileges, User credentials, Installed plugins, A vulnerable theme or Any prior access to the website. If the site is running an affected version, the vulnerable code is already present.

Researchers from Assetnote, part of Searchlight Cyber, discovered the issue and reported it responsibly through WordPress' HackerOne bug bounty program.

The vulnerability was first introduced into WordPress v6.9.0 back on December 2nd of 2025. And it's been carried forward since then. The v7.0.0 release inherited the new v6.9.0 vulnerability with its first release toward the end of May on the 20th of this year. Hopefully WordPress' forced update will have updated all vulnerable systems before the news of this vulnerability can draw attacks.

This is as bad a vulnerability as any we've seen from WordPress so anyone running the v6.9.x or v7.0.x releases should verify that they are now at least at v6.9.5, v7.0.2 or the latest beta 2 of v7.1. It's important. And a thanks and tip of my hat to our listener Simon Zerafa for bringing this to my attention.

Listener Feedback

Erfan Reed

Hi Steve and Leo, I have been tuning in to Security Now and TWiT since 2009, so when I heard your recent show featuring the Agent Smith persona for LLMs, I knew I had to reach out.

I have a simple open source project called mcp-speak that gives LLM agents their own voice and distinct personality. I originally built five personas for my MCP server, ranging from a sarcastic senior engineer to a tech priest. After listening to your episode, I couldn't resist adding an Agent Smith persona to the mix. The results are incredibly fun, especially when you configure the settings so the agent addresses you as "Mr. [Your Last Name]."

Beyond the novelty, it is genuinely useful for multitasking. On long-running operations, the LLM agent can simply speak up and notify you out loud whenever it needs input or finishes a task. The project runs locally on macOS and utilizes the native built-in `say` command for text-to-speech, so there is no need for external voice API keys. If you or your listeners want to check it out, the project page is right here:

<https://fellowgeek.github.io/mcp-speak/>

Thanks for the decades of great content, and keep up the awesome work!

I went over and took a look. It looks like Erfan did a nice job. You clone the repository and run his setup wizard with the command "python3 [setup.py](#)" and you're off. Erfan shows manual integrations for Google Antigravity, Claude CLI, Claude Desktop, Cursor IDE and Windsurf Editor. And he provides personalities for the Sarcastic Senior, the Eager Intern, the Existential Emo, the Pun Master, the Tech Priest, Agent Smith and a Gothic Poet.

Chris Gollner

Hi Steve, I've listened to Security Now since episode one, and it's funny how time passes. In that time, I got married, raised two amazing kids, and still listen to you and Leo every week. Hi, Leo.

Back then (1997/1998), I was a COBOL programmer. There was no such thing as vibe coding, and, to be honest, I don't know what that even is. Today, I find myself with ChatGPT 5.5 open on the left, Codex 5.6 Sol open on the right, and all the program specification documents in the Explorer window. I was taught to design first, think it through, and plan before writing a single line of code. It was good advice then, and it's good advice today.

GPT 5.5 lets me have discussions about what I'm trying to write and helps me write and review the specification. It really is amazing at doing all that grunt work—something I'd have given to a junior engineer, and work that I did myself decades ago.

Providing Sol with the specification blew my mind. It reviewed the specification, broke the project into milestones, and then broke those milestones into slices. Even now, while writing this, I can see C# code flicking across the screen in the background as it works on Milestone 1, Slice 6.

When it finds contradictions, collisions, or canon-breaking ideas, it challenges them and asks for an authoritative decision. I discuss these with GPT 5.5 and eventually respond to 5.6. It kind of reminds me of The Forbin Project, except I'm still a key component between the two.

Seeing my little app go from concept to a working program, with more and more features appearing as each slice and milestone progresses, really does leave me in a state of awe.

One day, this will be commonplace. But right now, this really is an amazing time to be alive. Cheers, Chris Gollner, Sydney, Australia

I know that this is the experience you've been having, Leo. And I thought that Chris beautifully captured the experience. I also think it's very cool that this podcast has been there through the past 20 years of Chris' life as he married, then fathered and raised a pair of kids. This podcast has really been something Leo. While many others have come and gone, this, one of the oldest and original, is still here and going strong!

Rich Ingersoll

Hi Steve, I oversee Vulnerability Management for a large Enterprise in NY. I'm still listening to the latest episode of Security Now, but your discussion of Cyber Shield really piqued my interest. I wanted to raise awareness to you about something our Cloud vendor is implementing.

We have a small(ish), but ever-growing presence in the cloud, so we are using Wiz to monitor that environment. Recently they introduced 2 of 3 agents that seem to accomplish what Cyber Shield is aiming for: Red/Green/Blue agents. Currently only 2 of these agents are available. The 3rd will be implemented soon. The end goal is to perform detection, investigation, and remediation at machine speed rather than human speed, as human response is too slow. To learn more, check out: <https://www.wiz.io/blog/introducing-wiz-agents>

Anyway - wanted to share some info from the trenches. If you decide to use this feedback, I would appreciate only my name being shared. Best Regards, Rich

Thanks for the pointer, Rich. Before I talk about Wiz, I wanted to mention how cool I think it is that the enterprise you work for has a "Vulnerability Management" role. So bravo to them for that and for also picking you to oversee it.

Our reporting frequently encounters the work of Wiz Security, so I was curious about this new offering of theirs. The page Rich linked to explains the roles of these three agents. They wrote:

Meet the Agents: Red, Blue, and Green

We built three specialized agents to operate across the entire security lifecycle. These aren't simple assistants — they are intelligent systems that can reason, investigate, and take action, grounded in the Wiz Security Graph.

The Red Agent is your AI-powered attacker. Red Agent reasons through application logic to uncover complex logic-driven vulnerabilities typically left hidden. It acts like a sophisticated

security researcher- but with AI speed and scale- reasoning about application behavior, adapting its approach in real time, and validating exploitable risks across your web applications and APIs. It empowers you to stay one step ahead of attackers.

The Blue Agent is your built-in threat investigator. When a threat is triggered, Blue Agent gathers evidence across cloud telemetry, runtime signals, and identity context to comprehensively investigate the threat and produce a clear verdict on its severity. It approaches threat investigation as a seasoned incident responder would, providing its full investigation logic so you can resolve threats with confidence and speed.

The Green Agent is your path to "zero criticals". Green Agent acts as a built-in investigation and remediation engine, continuously analyzing your highest-risk Issues to close the gap between detection and resolution. Like a seasoned security engineer, it synthesizes context from across Wiz - including the Security Graph, code-to-cloud relationships, identity ownership, and historical remediation patterns - to identify the true root cause of a risk and the safest, most effective resolution. Teams get environment-specific, step-by-step remediation guidance so fixes are durable.

Together, this team of agents form a continuous loop of validation, investigation, and resolution — all grounded in real context across your environment.

As Chris observed about the AI-enabled environment he is now coding in, someday this will all be commonplace but today it's an amazing time to be here and participating.

Rich's pointer to Wiz Security, who already has the first two of these three agents up and running and his reference to the UK's Cyber Shield plan which we talked about last week does give me pause to consider that perhaps having the UK bring up something like this won't be as far fetched as I thought. The scale of the thing would be massive... but if there's anything these AI systems are able to do with apparent ease, it's scale.

Greg Taylor

Hi Steve, I've seen this before at a Charles Schwab building where I worked for many years on backend trading systems. That's me there. There were 4 floors in the building but the stairs kept going. I guess this would have been the 5th floor? Nobody seemed to know.

Your friend and loyal listener, Greg Taylor



Bruce Behrens

Hi Steve, Love the podcast, longtime listener, SpinRite owner, etc.!

*I was listening to #1087 today after watching Yuval Noah Harari's video last night:
<https://www.youtube.com/watch?v=hBtVGwuJzpk>*

You and Leo were complaining about bureaucracy and bureaucrats. Interestingly, Harari's topic was bureaucracy. His take is that "bureaucracy is the machinery that lets strangers cooperate at scale. Bankers, lawyers, accountants, civil servants and religious authorities create trust by moving information through systems." And that "AIs are native bureaucrats." The implication is interesting. We shouldn't fear Claude the Terminator, we should fear Claude the bureaucrat.

The other question #1087 raised in my mind is whether, after 5 or 6 months of Mythos fixing all the code, there will be a need for Security Now!? Maybe Mythos will put you out of a job! ;)

Regards, Bruce

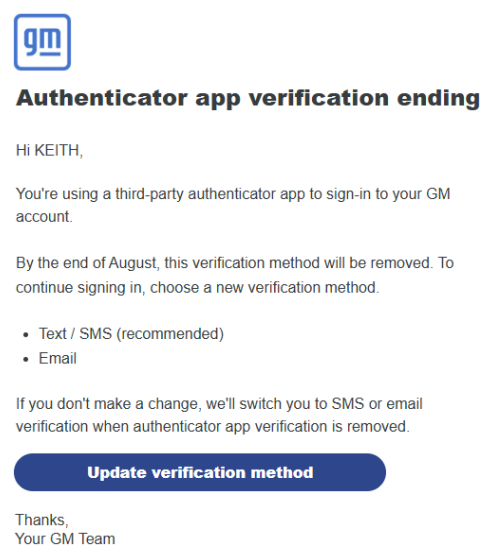
First of all, I love the notion of casting bureaucracy as "the machinery that lets strangers cooperate at scale." I think that's a very interesting observation. It makes sense to me on many levels.

Secondly, if after 5 or 6 months of Mythos and others fixing all of the code, there were to be no need for Security Now! I could not think of any better way to bid everyone a fond farewell. However, one lesson we've learned is that not all security mess ups are the result of software bugs; many, yes ... but certainly not all. Traditionally, we've been inclined to observe that there's always that "human factor" to screw things up. But now we've introduced a brand new and very wild card into the mix. I wouldn't be at all surprised to be observing a year or two from now that the "AI factor" will have become a new source of surprise. And in the security world, surprise is rarely a good thing.

Keith

I figure you may have already known about this, but in case you did not, General Motors has recently sent out an email stating that they will be removing the 2FA option I've been using with BitWarden from my account and forcing me to use either Text/SMS or Email.

Anyway, the email they sent is included below and if you use this just call me Keith.



That's really interesting. I wonder what the backstory is for this. I wonder whether they offered the use of second factor rolling 6-digit authentication to their subscribers in the interest of heightened security, but then had so many technical support calls from people who didn't know how to use it or were somehow becoming all tangled up that they just decided that, insecure or not, life would just be simpler without 2FA at all.

I recently had the experience of creating an account somewhere as part of the home moving effort we've been focused upon for the past few months. Whatever the site was, all they wanted was an email address. They sent an email with a button to click to verify. Never was any password requested or mentioned.

As we know, I've observed that since all forms of typical password recovery ultimately reduce to "prove that you're you by responding to the email we just sent you" — this solution is pretty much as secure as anything else. From this view, as I noted at the time, any password-based system is actually just a login accelerator. Using it allows the slower email loop system to be bypassed.

He Kai

I agree that we are headed into a whole different world. But let me suggest to you that the world might not be as uniformly rosy (with regard to software) as you suggest.

It is clear that AI can, when harnessed to do so, find and sometimes fix issues in both software design and software implementations. If the software of the future was roughly similar in size and scope to the software of today then, as AI reduces in cost over time, more and more CI/CD pipelines would adopt AI enabled review tools and software would dramatically raise its trustworthiness.

*But consider this: AI will also dramatically increase the **amount** of code in the world. My recently retired father, having no background in programming, built his own website with AI. He has no clear notion of what can go wrong when it comes to security, he doesn't have a CI pipeline, he doesn't even know the right questions to ask or how to evaluate the answers.*

I have a fear that security issues will become widespread as AI copies and pastes the mistakes of the past at speed and scale.

The comment our listener made that interested me the most was something I had never considered before, which is that AI-enabled code generation promises to dramatically increase the total amount of code in the world. Of course we absolutely know that's going to happen. Already, non-coders are using AI to create systems they could never have before. And existing coders are becoming far more productive. All of that is going to mean much more code. For what it's worth, I see that as a generally positive development for the world. The many things computers **could** do for people have, until this AI coding revolution, been completely out of reach of most people. They were limited to using what someone **else** designed and created. Now we're approaching a natural language interface that allows anyone to have a discussion with an AI about what it is that they would like to have their computer do for them and "*snap crackle and pop!*" this amazing genie we've created is able to turn their descriptive discussion into working code. It's beyond huge. It's utterly transformational. You go, Grandpa!!

A nefarious novel use for AI

Given how Large Language Model AI has proven to be so capable of discovering vulnerabilities in existing code, pretty much everyone has viewed the malicious abuse of AI through the lens of a classic arms race. With this view, the question is whether the good guys are going to be able to discover vulnerabilities, then patch and deploy less vulnerable code before the bad guys are able to discover their own vulnerabilities which will then allow them to develop exploits and attack the existing still-vulnerable code? In other words, who will be first to either fix or exploit the deployed vulnerabilities? It's only natural that this would be where everyone is focused.

But the old truism "*necessity is the mother of invention*" comes to mind when we learn that those ever-nefarious bad guys turned out to have an entirely different type of AI-solvable problem that no one had stopped to consider. As necessity would have it, AI has been proven able to provide massive leverage in an area that had never been considered before.

One thing that's interesting is that we've actually touched upon this problem that bad guys have faced in the past. We've wondered how ransomware baddies who arrange to download terabytes of victim data are able to make heads or tails of their plunder? And having revisited this previously open question, everyone listening now knows exactly how AI is being used not to help the bad guys penetrate a victim's network, but, instead, to help them understand the value of what they've obtained once terabytes of that victim's data has been exfiltrated. This is, indeed, "A nefarious novel use for AI".

The firm Glidepoint Security recently published their April-to-June, 2nd Quarter, 2026 report titled "Ransomware and Cyber Threat Insights". This 26-page report examined the many various aspects of the ransomware phenomenon we've previously covered. But their section titled "*AI is an Enabler, but Not How You Would Think*" addressed this entirely new aspect which exists at the intersection of a classic problem faced by ransomware perps and new LLM AI capabilities.

The sub-head of this section is titled: "*How Threat Actors are Using AI in Ransomware Negotiations*". They write:

Contemporary discourse around threat actors' usage of LLM AI ranges from legitimate concern by Defenders to outright fear, uncertainty, and doubt (FUD)-mongering by others. Since the AI boom began in late 2022, AI innovation has moved at an unprecedented pace, making it difficult to separate the potential from the actual in real time. It is imperative to isolate signal from noise by grounding claims on the subject in empirical data.

***FulcrumSec**, a data extortion group we first identified in late 2025, has deployed LLMs operationally during ransom negotiations involving the theft of a victim's highly complex production database. GRIT (GuidePoint Research and Intelligence Team) has observed what we assess to be the processing of exfiltrated data by the group through an unidentified LLM to generate step-by-step instructions for linking user identities across several databases. We base this assessment on the analytical output's complexity relative to FulcrumSec's known baseline capability, as well as the precision of the threat actor's language during negotiations.*

In other words, these GRIT guys have been carefully watching and documenting FulcrumSec's

activities for the past year. So they know that these bad guys would be incapable of making either heads or tails out of the download of a large raw database. But at the same time they know that a contemporary AI agent could do this without breaking a token. They wrote:

Due to the complexity of the database schema, this analysis of a victim's data would have been implausible without either deep internal knowledge of the victim's database architecture, a substantial period of focused human attention, or AI assistance. Given the abbreviated time in which the negotiations occurred, we find it unlikely that a threat actor would have the capacity to fully untangle a complex database schema given the time available. We have included a re-creation of the usage of AI during the negotiation:

We understand it is a lot to wrap one's head around. This is a big one.

Regarding how we linked identities across the databases, the short answer is: your own schema makes it trivial for us to do so. Nearly every table in both databases shares a single key. That one key links (mostly) everything. This is by design, or your own analysts wouldn't be able to work with the data. Here's a more technical walk-through of how it works in practice, even when some values were hidden or hashed in your production databases:

Step 1: Start with [PRIMARY_IDENTIFIER]. Your staging tables contain this [PRIMARY_IDENTIFIER] in plain text. The main source is a table that stores about [X] million unique customer names, dates of birth, and home addresses. Every row has a [LINKING_KEY] attached to it. That's the starting point.

Step 2: Follow the [LINKING_KEY] to everything else. That same [LINKING_KEY] appears in dozens of other tables across your databases. One simple database query connects a single [PRIMARY_IDENTIFIER] to driver's licenses and state IDs, bank account and routing numbers, email addresses, phone numbers, and so on. Each of those is one query away from the [PRIMARY_IDENTIFIER].

No guesswork is required: the [LINKING_KEY] is a direct link your own engineers built into the schema.

Step 3: The hashed [PRIMARY_IDENTIFIERS] weren't real protection. Some tables stored [PRIMARY_IDENTIFIERS] as cryptographic hashes (SHA-256) instead of plain text. But [PRIMARY_IDENTIFIERS] are only [X] digits with only roughly [X] million possible values. A single computer can hash every possible [PRIMARY_IDENTIFIER] in under [X] minutes, producing a lookup table that maps every hash back to the original number. We reversed millions of them in minutes. If these had been hashed in a cracking-resistant algorithm, we would not have even bothered trying — we would have needed a datacenter's worth of compute power running full blast for months to make a real dent in them. Impractical. It's worth noting that user passwords were properly hashed, so again, your team knew how to do this, but chose not to apply it to other data.

Step 4: The encoded [PRIMARY_IDENTIFIERS] were even weaker. Your tables stored [PRIMARY_IDENTIFIERS] with a simple character substitution — that is, each character shifted by a fixed amount (e.g., "1" becomes "9", "2" becomes ":"). A one-line script reversed [NUMBER] of these instantly. (This is known as a Caesar cipher and it is from ancient Rome. It is not secure.)

What this means functionally is that starting from any single customer, one query produces a

complete identity package: [PRIMARY_IDENTIFIER], name, date of birth, address, driver's license, bank account, email, phone, employer, income, credit score, security question answer, password hash, full loan history, and, for hundreds of thousands of your customers, verbatim notes about the most difficult moments of their lives. The data warehouse was designed to work this way. We're happy to answer any more questions at your request.

GuidePoint's feeling is that there's no way this FulcrumSec group can have possibly performed all of this reverse-engineering work on the downloaded database material given the time they observed. They had to have used the speed offered by AI.

GuidePoint concludes their examination of this specific FulcrumSec event by writing:

Additionally, FulcrumSec used LLM-generated language during their negotiation with the victim, communicating in language clearer and more precise than any typically observed for non-native English-speaking threat actor groups. The language helped the group anchor their position and drive negotiations from their side, in effect saying: "We know what we have taken, here it is, and this is why we have set the ransom at this amount." This is markedly different from most threat actor negotiations, where operators commonly use open-source platforms like Crunchbase or ZoomInfo to establish ransom amounts based on market data. By applying LLM capabilities — analytically rather than generically — FulcrumSec established a firm negotiating stance from which they had little incentive to diverge.

Okay. There's the first of two concrete examples. Next they look at a group known as "DragonForce", writing:

Where FulcrumSec used LLMs to process and weaponize data, DragonForce demonstrates a second and equally significant use case: deploying LLMs to manufacture plausible pressure that would otherwise require capabilities the group does not have.

DragonForce is an Established ransomware-as-a-service (RaaS) group previously covered by GRIT (see our Q2 2025 report). Building on that prior analysis, GRIT has observed DragonForce incorporating AI and LLMs into its operations, a meaningful shift from its earlier tradecraft. Most notably, during negotiations and in advertisements for potential affiliates, the group has claimed to have legal counsel on staff. The statement, which is almost certainly false, is designed to pressure victims by implying that DragonForce has insight into a victim's reporting requirements and legal exposure arising from the data leak.

The notion of a criminal ransomware group retaining lawyers fully versed in international data requirements is absurd — until you realize the 'lawyer' is an LLM. For criminal purposes, it doesn't matter if the claim is true; it only matters if it sounds plausible. If there's one thing LLMs are good at, it's making a wide range of statements sound entirely plausible.

So what does this mean? AI and LLM use gives threat actors a structural advantage in negotiations. They significantly reduce language barriers, increase negotiation professionalism, and amplify available psychological pressure to bear against the victim. Historically, analysts could use imperfect non-native English as a soft attribution marker of adversary geographic location. Even tools like Google Translate would leave tell-tale signs. But contemporary LLM

use reduces or even eliminates that signal entirely. This is not a marginal development. Attribution confidence decreases, negotiation dynamics shift toward threat actors, and the gap between sophisticated and unsophisticated groups narrows in ways that make victim preparation critically more important.

More broadly, increased threat actor AI/LLM use reinforces the efficacy of the RaaS business model. Conti pioneered the RaaS model, structuring affiliate programs and playbook-driven syndicate operations that set the template that's now being further professionalized and automated by AI tooling. AI and LLMs allow less sophisticated and non-native English speaking groups to approach negotiations in a more professional manner, establishing negotiations with unprepared victims on their terms.

GRIT will revisit this topic throughout the year to assess the question: will threat actors continue refining AI/LLM integration into their processes, will it plateau, or will the use of AI/LLMs be more limited to specific groups? GRIT anticipates threat actors will continue to streamline LLM usage in the near-term, primarily through the two vectors: negotiation communications and exfiltrated data analysis. More complex, sophisticated, or novel adoption of AI/LLMs will almost certainly be more limited but may "trickle down" in the long-term.

Next are the next steps for defenders?

Threat actor adoption of AI/LLM tooling raises the floor for negotiation sophistication across the board. It reinforces organizations' need for cybersecurity insurance, legal counsel, and an understanding of the data present in their environment. It also suggests that negotiations should be conducted by trained professionals. While threat groups do have some predictable behavior, individual operators are criminals who may act erratically, causing dire consequences for the victim organization. Engaging qualified professionals gives organizations a clear understanding of threat actor playbooks and current behavior, enabling them to distinguish routine bluffs from credible threats. Expert legal counsel is also essential for understanding reporting requirements and potential legal ramifications. A mature and up-to-date incident response plan can assist with the coordination of all these factors.

Okay. So that was GuidePoint's example of two real world threat groups. Their report was a bit more sanitized than I was hoping for. So I did a bit more digging to find some additional reporting on FulcrumSec, that first group GuidePoint discussed. The reporting I found added:

As an example of the consequence of this group's use of AI, in June of last month, FulcrumSec reached out to [DataBreaches.Net](#) regarding their compromise of the Danish pharmaceutical company Novo Nordisk, the maker of Wegovy a well-known Semaglutide GLP-1 agonist drug. FulcrumSec claimed to have stolen 1.3TB of data containing 700,717 files.

I'll interrupt briefly to note that this is a textbook example of wondering what to do with the presumed treasures that were just plundered from a victim. On the one hand it's "Hot damn! We just sucked out 700,717 individual files with an aggregate size of 1.3 TERAbytes of data!" But now what? Hopefully there's some really juicy data that we can use for blackmail extortion but, uh. where would it be, exactly, hiding among that mass of more than .7 MILLION individual files? Continuing...

FulcrumSec said it had captured valuable intellectual property including five publicly

undisclosed drug programs, in-development drug and RNA delivery programs and private AI models for particular medical and drug discovery purposes.

The group told DataBreaches that it used a team of AI agents to analyse those private models and that it believes the stolen data could save competitors three to five years of program development. Its initial ransom demand to Novo Nordisk was for \$25 million USD.

The information about the intellectual property FulcrumSec stole was coupled with a description of Novo Nordisk's security posture, which the group claimed was "absolutely catastrophic" and "boggles the mind." To us, this sounds like FulcrumSec is attempting to frame the incident in a way that would have any class action lawyer salivating. It wouldn't be the first time a data breach has resulted in a lawsuit, so presumably this is part of FulcrumSec's extortion pitch. Their modus operandi also includes using AI to generate detailed reports which it then provides to threat researchers and journalists, nicely formatted, complete with logo and all, in order to apply more pressure to victims.

For example, after compromising the technology company Avnet in October last year, the group gave the vx-underground X account a report on the breach. According to vx-underground, the group provided "an autobiography, a breakdown of the data they possess, their motives for the compromise, information on their logo design (and why their logo was chosen), a complete stolen file listing of the compromise, a breakdown of the files (what it is, what they are, what they contain) and images of the files."

vx-underground said the group had done "every bit of research and write up for us." To add insult to injury, FulcrumSec claimed it had used an OpenAI key it had stolen from the victim to pay for ChatGPT to summarise the victim's own data.

So... I started out noting that "necessity is very often the mother of invention." Since none of us are on the inside of any of these ransomware gangs we have a difficult time imagining what their problems might be. So the world comes up with "Hey! They're probably going to use AI just like software publishers will, to discover previously unknown vulnerabilities and then use those to compromise systems.

While that will doubtless be one use, the evidence suggests that the bad guys don't need new ways of getting into other people's networks as much as they need help after the data has been successfully exfiltrated and is in their hands. They need AI's help with determining the value of what they just grabbed and then help negotiating with the data's legal owners who almost certainly speak a language that they do not. Appearing tough, competent and knowledgeable is every bit as important after the theft as obtaining the stolen goods was in the first place. They have ZERO interest in the data they've just obtained. Its entire value to them lies in what cold hard cash they can trade for destroying that data that they now hold. And for that, AI has been the best thing to happen for them . . . ever.

